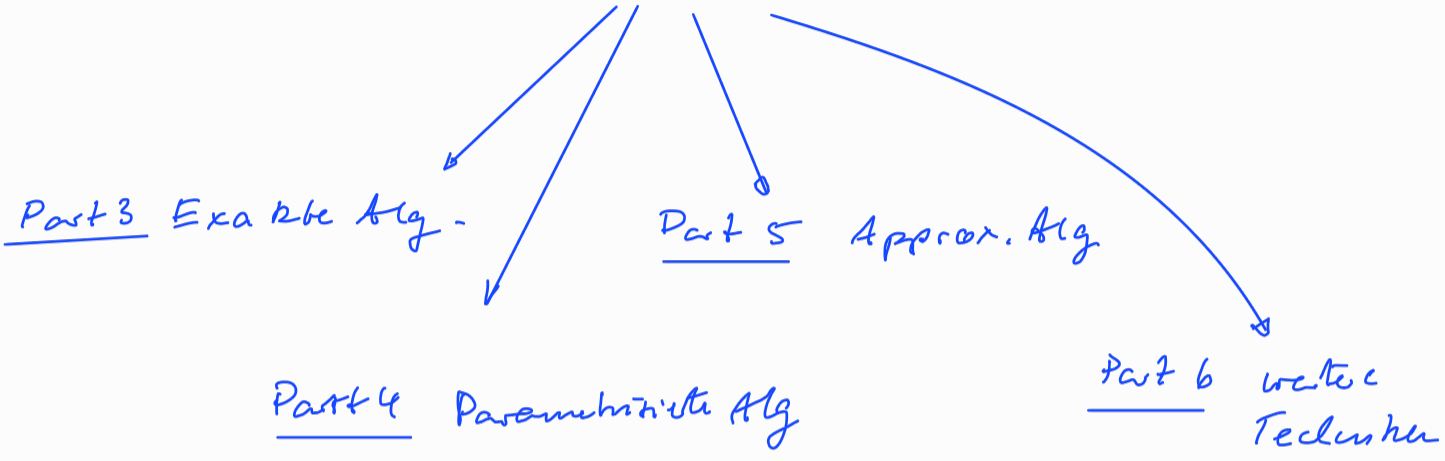
Fahrplan

2.3

Part 1 TM, Algorithmen & Entscheidbarkeit

Part 2 Einfach vs schwere Probleme & NP vollständigkeit



[siehe slides!] =>

2.2 NP-vollständigkeit in der Praxis

wdh: $P = \{$ Entscheidungsprobleme die mittels determ. Algor/TM in polynomieller Zeit gelöst werden können

$NP = \{$ Entscheidungsprobleme die mittels nicht-determ. Algor/TM in polynomieller Zeit gelöst werden können

$= \{$ Entscheidungsprobleme die mittels determ. Algor/TM in polynomieller Zeit verifiziert werden können

Entscheidungsproblem D ist NP vollständig

wenn:
- $D \in NP$ (zeige, daß gegebene Lsg in polyzeit verifizierbar werden kann)
- $\forall D' \in NP: D' \leq_p D$, hierbei steht $\leq_p$ für polyn. zeit reduktion

(• nimm bekanntes NP-vollst. problem
• gebe Transformation beliebiger Instanzen $I' \in D'$ in $I \in D$ an od:
  • Transformation geht in poly. zeit
  • I' JA-instanz $\Leftrightarrow$ I JA-Instanz.)

Thm (Cook-Levin) SAT ist NP vollständig.

wir haben gesehen 1-& 2-SAT sind polyzeit lösbar.

was ist mit 3-SAT?

3SAT: is SAT instance s.t. clause consists of exactly 3 literals

$\varphi = C_1 \wedge C_2 \wedge C_3 \wedge \dots \wedge C_n$, $|C_i| = 3$ $1 \le i \le n$.

Q: ∃ truth-assignment to variables s.t. φ = true?
[φ is satisfiable]

Thm: 3SAT is NP-complete

proof: 3SAT ∈ NP: guess solution = assignment of 0/1 to Boolean variables

check if each clause is "true".

works in poly-time ✓

3SAT is NP hard: (so-far we only know that SAT is NP-C so let's try this)

show $SAT \le_p 3SAT$

Let φ be an arbitrary instance of SAT.

Let nr of clauses of φ be m: $\varphi = C_1 \wedge C_2 \dots \wedge \dots C_m$

IF all clauses C_i have size $|C_i| = 3$, then there is nothing to transform.

but there are instances of SAT where $|C_i| \neq 3$

We consider the following cases for each C_i: $|C_i| = 2$
$|C_i| = 1$
$|C_i| > 3$

To show that φ is true
it suffices to show that each clause is true!

$|C_i| = 2$: introduce new variable u, remove $C_i = (x \vee y)$

eg. $C_i = (x \vee y)$

& 2 new clauses $C_i' = (x \vee y \vee u)$

$C_i'' = (x \vee y \vee \bar{u})$ ↑ not(u)

to show: C_i true $\Leftrightarrow$ $C_i' \wedge C_i''$ true.

IF C_i true $\Rightarrow$ at least one of x & y must be true

$\Rightarrow$ C_i' & C_i'' true

IF $C_i' \wedge C_i''$ true $\Rightarrow$ C_i' true & C_i'' true

IF $u = 1 \Rightarrow \bar{u} = 0 \Rightarrow$ in C_i'': $x = 1$ or $y = 1$

$\Rightarrow C_i$ true

IF $u = 0 \Rightarrow$ in C_i': $x = 1$ or $y = 1 \Rightarrow C_i$ true. ✓

$|C_i| = 1$: introduce new variable u,

remove $C_i = (x)$

add $C_i' = (x \vee u)$, $C_i'' = (x \vee \bar{u})$

[Ex.: C_i satisf. $\Leftrightarrow$ $C_i' \wedge C_i''$ satisfiable]

add new variable v

& replace C_i' & C_i'' by

$C_i^{1'} = (x \vee u \vee v)$

$C_i^{2'} = (x \vee u \vee \bar{v})$

$C_i^{1''} = (x \vee \bar{u} \vee v)$

$C_i^{2''} = (x \vee \bar{u} \vee \bar{v})$

} in this way the clauses are linked and one of uv, $u\bar{v}$, $\bar{u}v$, $\bar{u}\bar{v}$ must false $\Rightarrow$ x must be true!

[Ex. C_i satisf $\Leftrightarrow$ $C_i^{1'} \wedge C_i^{2'} \wedge C_i^{1''} \wedge C_i^{2''}$ satisf.]

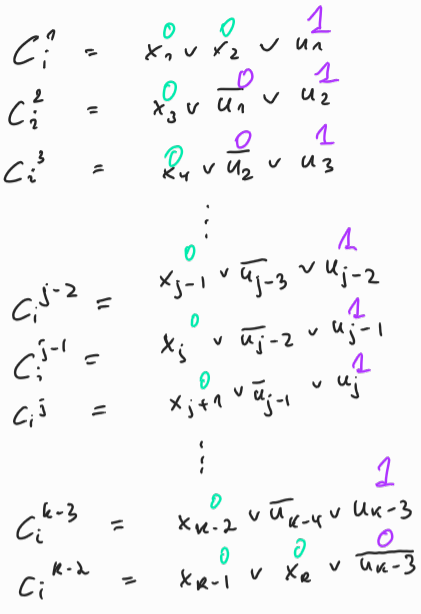
$|C_i| = 3$ nothing to do.

$|C_i| > 3$, let $C_i = (x_1 \vee \dots \vee x_k)$, where x_i are literals

add new variables $u_1, u_2 \dots u_{k-3}$
remove C_i & add new clauses:

$C_i^1 = x_1 \vee x_2 \vee u_1$
$C_i^2 = x_3 \vee \bar{u}_1 \vee u_2$
$C_i^3 = x_4 \vee \bar{u}_2 \vee u_3$
$\vdots$
$C_i^{j-2} = x_{j-1} \vee \bar{u}_{j-3} \vee u_{j-2}$
$C_i^{j-1} = x_j \vee \bar{u}_{j-2} \vee u_{j-1}$
$C_i^j = x_{j+1} \vee \bar{u}_{j-1} \vee u_j$
$\vdots$
$C_i^{k-3} = x_{k-2} \vee \bar{u}_{k-4} \vee u_{k-3}$
$C_i^{k-2} = x_{k-1} \vee x_k \vee \bar{u}_{k-3}$

main idea: link the literals $x_1 - x_k$ in clauses s.t.
C_i true $\Leftrightarrow C_i^1 \dots C_i^{k-2}$ true



to show: C_i satisfiable $\Leftrightarrow C_i^1 \dots C_i^{k-2}$ satisf.

„$\Rightarrow$" C_i satisf. $\Rightarrow$ at least one $x_j = 1$

Set $u_r = 1$, $r \le j-2$
$u_r = 0$, else

$\Rightarrow$ all $C_i^1 \dots C_i^{k-2}$ are true.

„$\Leftarrow$" assume $C_i^1 \dots C_i^{k-2}$ true

Proof by contradiction,
ie, assume C_i is not satisf.

$\Rightarrow x_1 = x_2 = \dots = x_k = 0$

$\Rightarrow$ in C_i^1: $u_1 = 1$

$\Rightarrow C_i^2$: $x_3 = \bar{u}_1 = 0 \Rightarrow u_2 = 1$

$\Rightarrow u_i = 0 \quad \forall i$

$\Rightarrow C_i^{k-2}$: $x_{k-1} = x_k = 0$
& $\bar{u}_{k-3} = 0$

$\Rightarrow C_i^{k-2}$ not true ↯

$\Rightarrow C_i$ satisf $\Leftrightarrow C_i^1 \dots C_i^{k-2}$ satisf.

[Ex: transformation can be done in poly-time]

$\Rightarrow$ SAT $\le_p$ 3SAT $/ \square$

Wir sieht daß jetzt mit anderen Problemen aus, wie etwa graphprobleme?
⇒ müssen bspw aus 4 graphen bauen.

Vollst graph $G = (V, E)$ ist unger. graph od $\forall u, v \in V$ mit $u \neq v$: $uv \in E$.

Abk: $K_{|V|}$: K_1 •, K_2 •—•, K_3 △, K_4  , ...

CLIQUE problem

Input: undirected graph $G = (V, E)$, integer $L \in \{1 \ldots, |V|\}$

FRAGE: exists $H \subseteq G$ st $H \cong K_r$, $r \geq L$?

Theorem: CLIQUE is NP-vollständig

Proof: CLIQUE $\in$ NP [Exercise]

3SAT $\leq_P$ CLIQUE

Let φ be an instance of 3SAT,

$\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_k$, $C_i = \{ l_1^i \vee l_2^i \vee l_3^i \}$

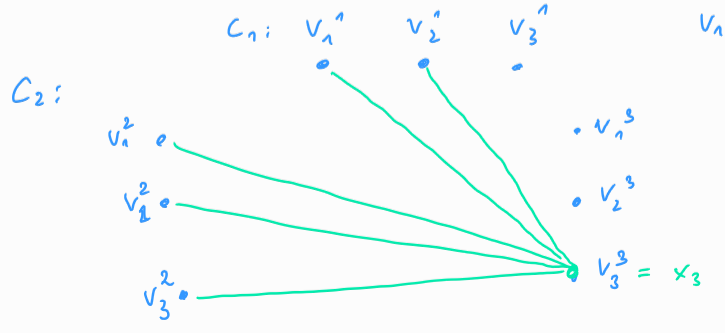
Need an instance of CLIQUE, i.e., a graph G.

Construct $G_\varphi = (V, E)$ as follows:

1) f.a. C_i add 3 vertices v_1^i, v_2^i, v_3^i
and "associate" v_r^i with literal l_r^i

2) add edge $v_r^i v_s^j$ if (i) $i \neq j$
(ii) corresp. literals l_r^i & l_s^j are not negations of from each other, i.e.
$l_r^i = x$, $l_s^j = \bar{x}$
resp $l_r^i = \bar{x}$, $l_s^j = x$ not allowed.

Exmpl $\varphi = (x_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$
($C_1 \wedge$ $C_2 \wedge$ $C_3 \wedge$)



$v_1^1 = x_1$, $v_2^1 = \bar{x}_2$, $v_3^1 = \bar{x}_3$
(analog for all other)
only edges from v_3^3 are shown.

to show: φ satisfiable $\Leftrightarrow$ G_φ has clique of size k

"$\Rightarrow$" if φ is satisfiable $\Rightarrow$ each C_i has at least one true literal l_r^i

$l_r^i \hat{=} v_r^i$ in G_φ

take from each C_i one true literal and save corresponding vertices in V'

$\Rightarrow V' \subseteq V, \quad |V'| = k$

let $v_r^i, v_s^j \in V' \xrightarrow[\text{by construction}]{(i)} i \neq j$

& corresp. literal l_r^i, l_s^j are not negations of each other, since they are both true.

$\xrightarrow[\text{by construction}]{(ii)}$ in G_φ exists edge (v_r^i, v_s^j)

$\Rightarrow$ $G_\varphi[V']$ is complete subgraph with k vertices ✓

"$\Leftarrow$" Assume $\exists\, V' \subseteq V, |V'| = k$ st $\forall\, u, w \in V' : (u,w) \in E$.

Since G_φ does not contain edge between vertices corresponding to the same clause

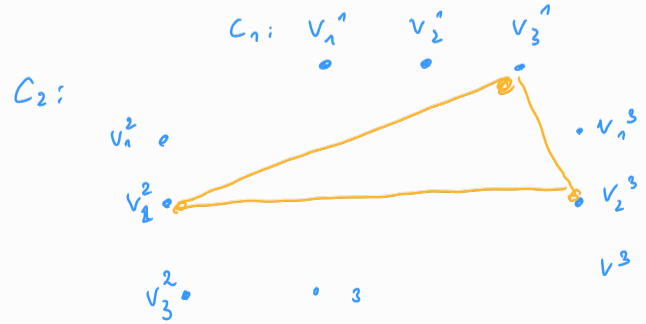
$\Rightarrow$ V' contains exactly 1 vertex of each clause

$\Rightarrow$ for each $v_r^i \in V'$ set literal $l_r^i = 1$

$\Rightarrow$ each C_i true $\Rightarrow$ φ satisfiable.

Exmpl $\varphi = (x_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$

$C_1 \supset 1$ $\quad C_2 \supset 1$ $\quad C_3 \supset 1$



$v_1^1 \hat{=} x_1$, $v_2^1 = \bar{x}_2$, $v_3^1 \hat{=} \bar{x}_3$

(analog for all other)

eg. k_3

Reduction can be done in polyn. time [Exerc]

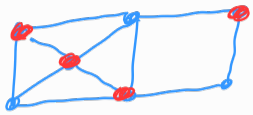
$\Rightarrow$ 3SAT $\leq_p$ CLIQUE $\square$

Vertex-Cover Problem (VCP)

input: undirected graph $G = (V, E)$, integer L

question: Does G have a vertex cover of size $k \leq L$, i.e.,
is there a subset $C \subseteq V$
s.t: (i) $|C| = k \leq L$
(ii) $C \cap e \neq \emptyset \quad \forall e \in E$

Exmpl



VC of size 4

Theorem: VCP is NP-complete

proof: VCP $\in$ NP [Exercise]

VCP NP-hard, by reduction from CLIQUE

we show: $G=(V,E)$ has CLIQUE of size k
$\Leftrightarrow$ $\bar{G}$ has VC of size $|V|-k$

Let $V' \subseteq V$ s.t. $G[V']$ forms a clique of size k
& put $C = V \setminus V'$
Assume, for contradiction, C is not VC of $\bar{G}$.
$\Rightarrow$ $\exists$ edge $(u,v) \in E(\bar{G})$ s.t. $u, v \notin C = V \setminus V'$
$\Rightarrow$ $u, v \in V'$
but since $(u,v) \in E(\bar{G}) \Rightarrow (u,v) \notin E(G)$ $\lightning$

Let $C = V \setminus V'$ be a VC of $\bar{G}$
$\Rightarrow$ $\forall (u,v) \in E(\bar{G})$: $u \in V \setminus V'$ or $v \in V \setminus V'$
$\Rightarrow$ at least one of the vertices u & v is not in V'
&
if $x, y \in V' \Rightarrow x, y \notin V \setminus V'$ $\xrightarrow[\text{VC of } \bar{G}]{C = V \setminus V'}$ $xy \notin E(\bar{G})$
$\Rightarrow$ $\forall x, y \in V'$: $xy \in E(G) \Rightarrow G[V']$ is complete graph on $|V|-k$ vertices

reduction works in polytime [Exns]

$\Rightarrow$ CLIQUE $\leq_p$ VCP

$\square$

Wir hatten das Problem kürzester Wege zu finden
in einem gewichteten Graphen (G, w).
Für $w(e) = 1 \quad \forall e \in E$ oder konservative Gewichtsfkt (keine neg.-gew. Kreise)
ist das problem „einfach" (= polynomiell lösbar).

Wie ist das aber mit allg. Gewichten?
Um dies zu verstehen betrachten wir zuerst das folgende Problem.

Sei $G = (V, E)$ ein gerichteter Graph, d.h. $E \subseteq V \times V$
ein uv-Weg, der jeden Knoten genau einmal besucht
heißt Hamiltonisch

PROBLEM	HAM_PFAD
Input:	gerichteter Graph $G = (V, E)$
Frage:	Gibt es einen Hamilt. Pfad in G?



Theorem: HAM_PFAD ist NP vollständig.

Beweis: $\in$ NP, da man für gratene Lsg (=Pfad) in polyzeit
prüfen kann ob jeder Knoten auch tatsächlich
genau einmal vorkommt
$\Rightarrow \in NP$ ✓

NP-hard?

Wir zeigen $3SAT \leq_p$ HAM_PFAD.

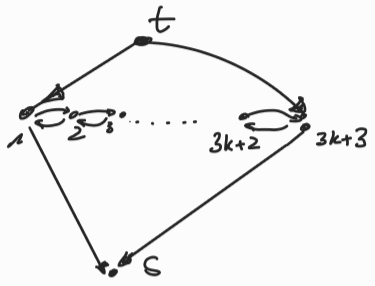
also sei $\varphi = C_1 \dots C_k$, $C_i = (a_i \vee b_i \vee c_i)$ mit Literalen a_i, b_i, c_i

eine beliebige Instanz von 3-SAT.

Die Boolsch. Variablen seien $\{x_1 \dots x_c\}$.

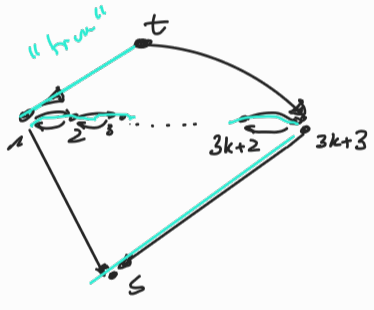
Wir konstruieren jetzt schrittweise einen Graphen, den wir aus einzelnen Gadgets (= Bausteine) zusammensetzen.

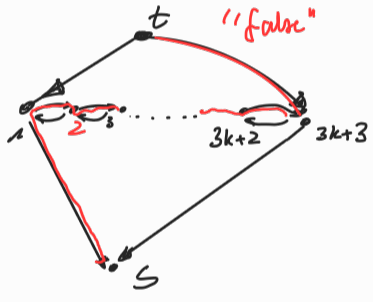
1. $\forall$ Variablen x_i baue



G_{x_i}

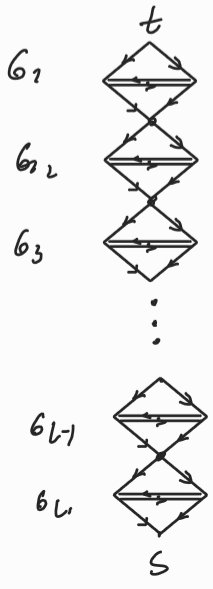
je nachdem ob man "TRUE" or "FALSE" wählt ist der Ham. Pfad in diesem Gadget eindeutig bestimmt:





Wir fügen diese Gadgets „  " wie folgt zusammen:

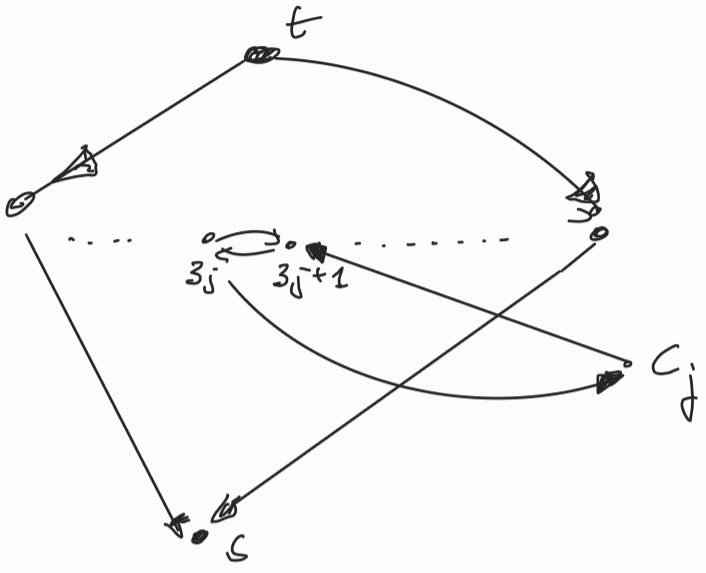
G_i



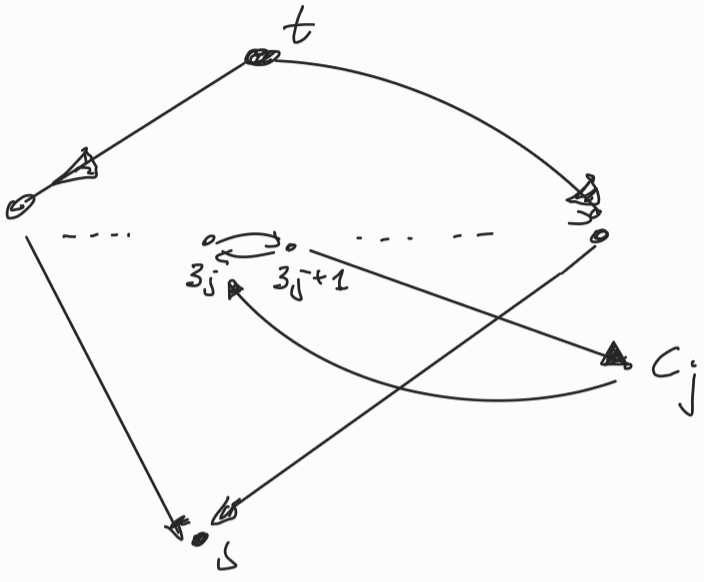
↙ gibt Ham. Pfad!

wir werden jetzt Variablen-Gadgets mit entsprechenden Klauseln identifizieren also C_j identif. mit x_i gadget wenn x_i bzw $\overline{x_i}$ in C_j vorkommt.

wie folgt:



wenn $x_i \in C_j$



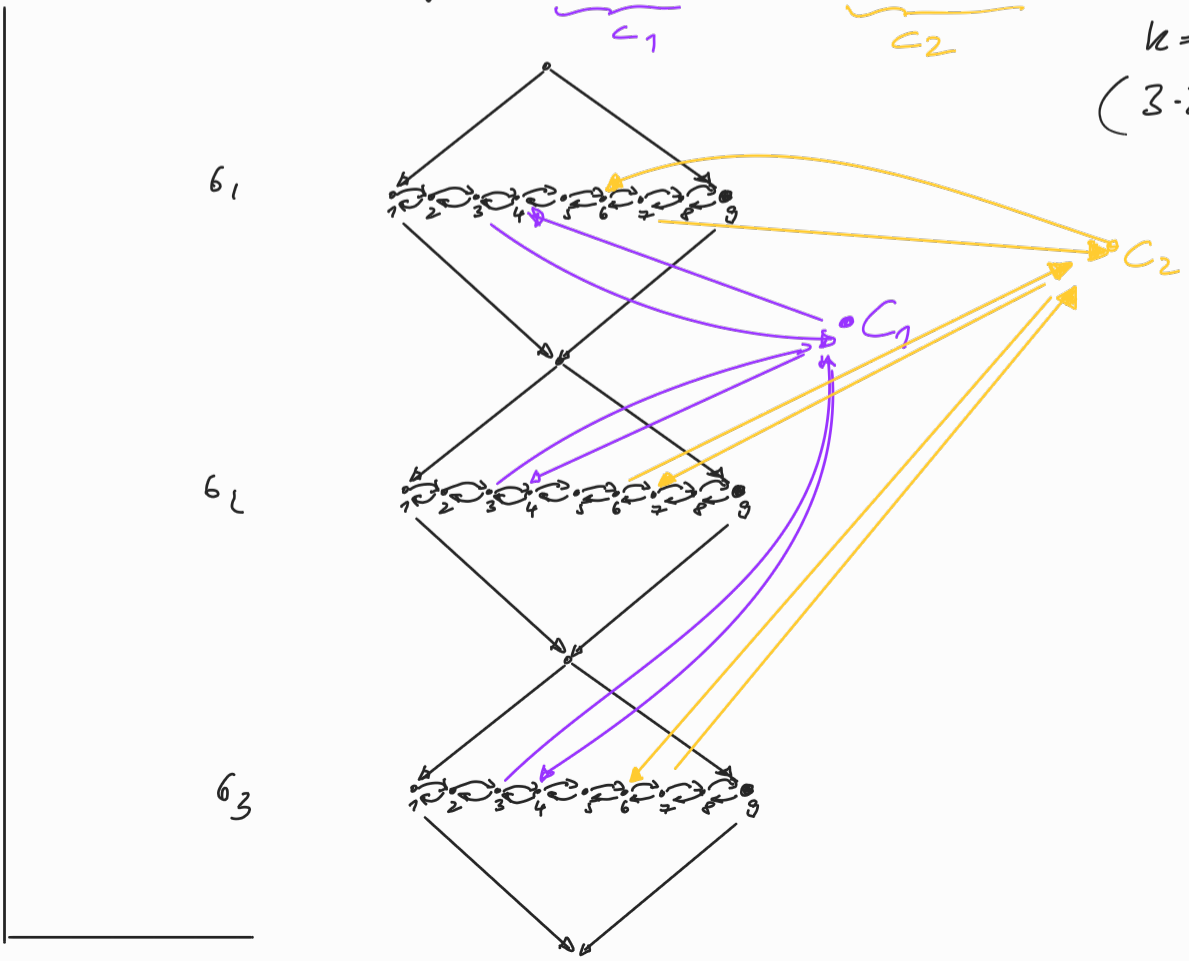
wenn $\overline{x_i} \in C_j$

konkretes Bsp für $\varphi = (x_1 \vee x_2 x_3) \wedge (\overline{x_1} \vee x_2 \vee x_3) = C_1 \wedge C_2$

C_1 C_2

$k = 2$

$(3 \cdot 2 + 3 = 9)$



Zeigen jetzt: φ erfüllbar $\Leftrightarrow$ G hat Ham-Pfad.

„$\Rightarrow$" Sei φ erfüllbar.

wenn x_i = TRUE nimm in Teilgraph G_i von G die Kanten $t \to 1 \to 2 \to \dots \to s$

x_i = FALSE nimm in G die Kanten $t \to 3k+3 \to \dots \to s$

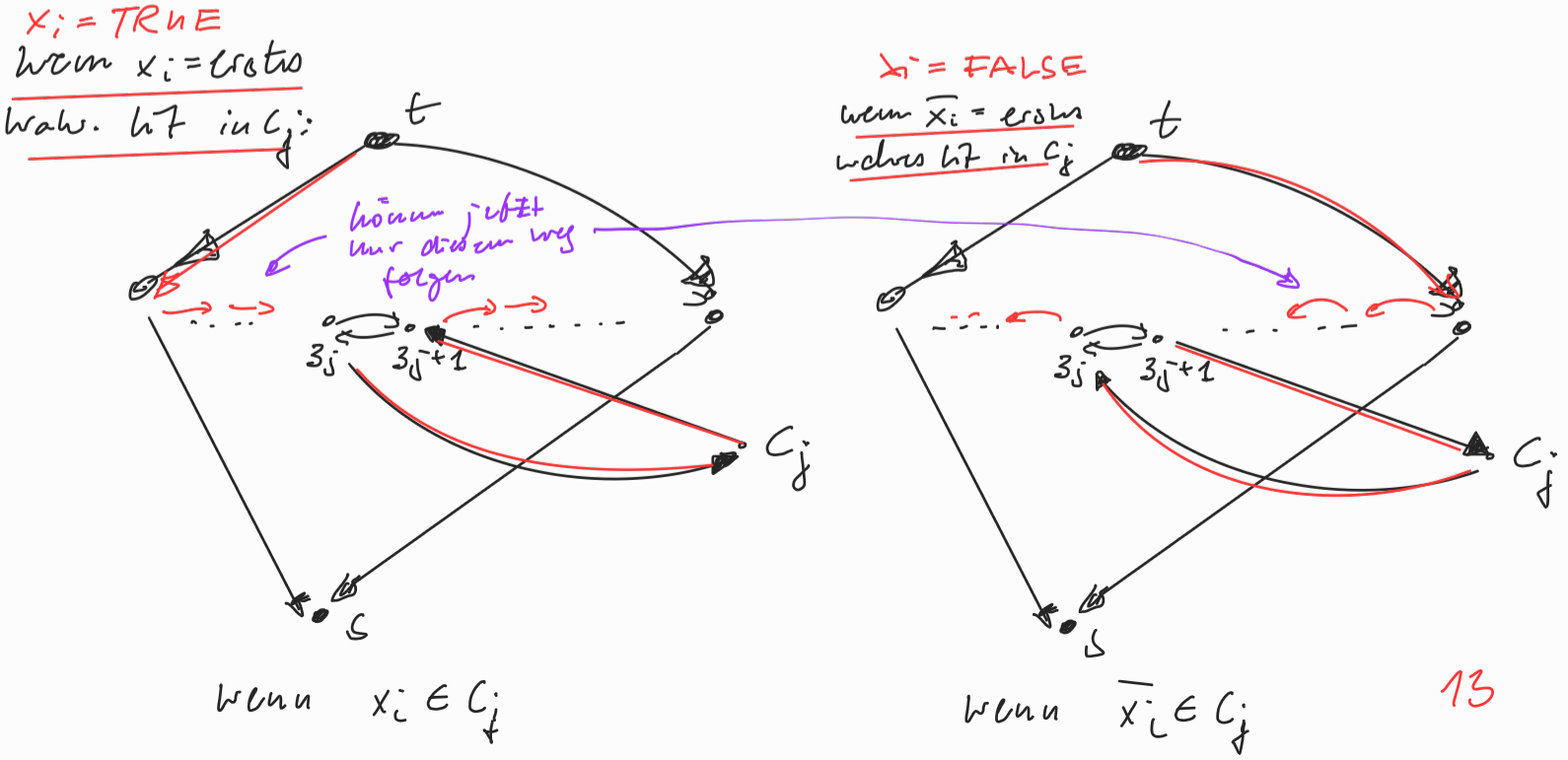
Beobachtung: jedes C_j ist der Form: $(a_j \vee b_j \vee c_j)$
da φ erfüllbar $\Rightarrow$ mind 1 Literal von a_j, b_j, c_j ist TRUE

wir können bel. ordnung auf jedem C_i festlegen. o.B.d.A $a_j < b_j < c_j$

bzgl diese Ordnung betrachten wir jetzt immer

das erste wahre Literal

(dieses Literal entspricht x_i or $\overline{x_i}$)



⇒ In beiden Fällen wird C_j besucht & danach nie wieder.

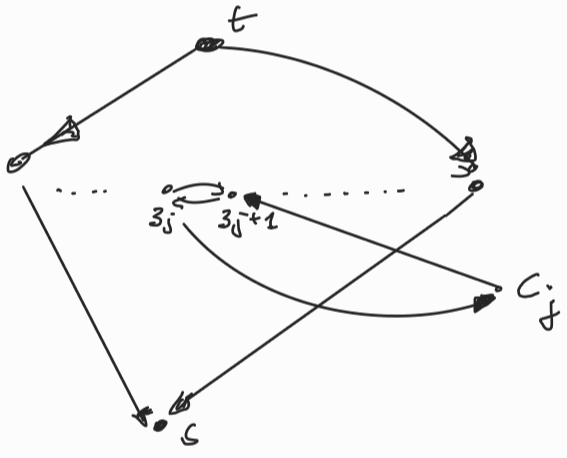
jedes C_j hat welches Lit da φ erfüllbar ⇒ C_j wird mind 1 mal besucht. ⇒ C_j genau einmal besucht.

alle anderen Knoten $\neq C_1 \dots C_k$ werden auch genau einmal besucht.

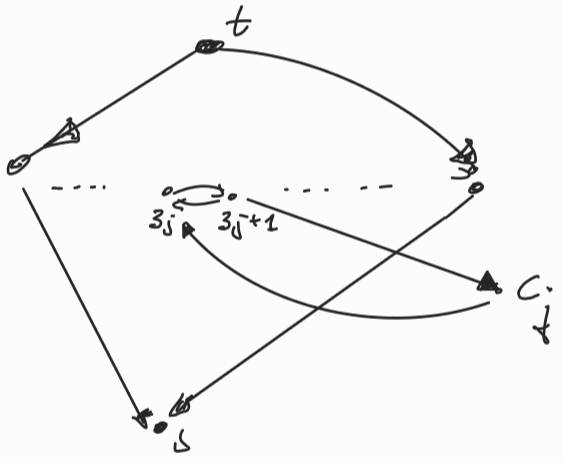
⇒ exist ham. Pfad in G.

„⇐" G hat Ham.-Pfad. P

⇒ alle C_j werden genau einmal besucht. über ein gadget G_i:

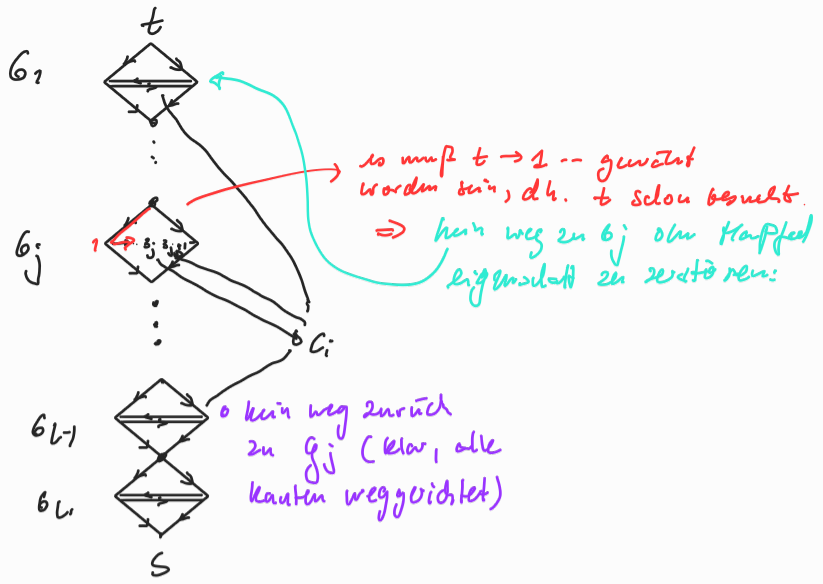


wenn $x_i \in C_j$



wenn $\bar{x}_i \in C_j$

Entweder über Kante $3_j \to C_j$, dann muß aber auch die Kante $C_j \to 3_j^-+1$ in P enthalten sein, sonst würde man in anderes G_r springen & käme nicht mehr zum Knoten 3_j+1 zurück.



Analog, wenn $3_j+1 \to C_i$ in P, ist, muß danach die kante $C_i \to 3_j$ in P anstehen.

Also in P gibt es immer kanten der Form

$3_j+1 \to C_i \to 3_j$ ⇒ setze x_j auf FALSE

[Fall $\overline{x_j} \in C_i$ ⇒ C_i = TRUE]

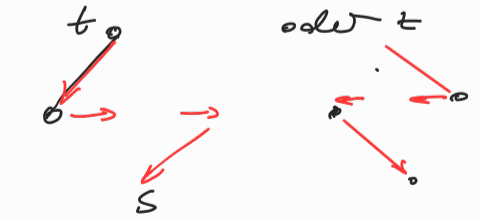
oder $3_j \to C_i \to 3_{j+1}$ ⇒ setze x_j auf TRUE

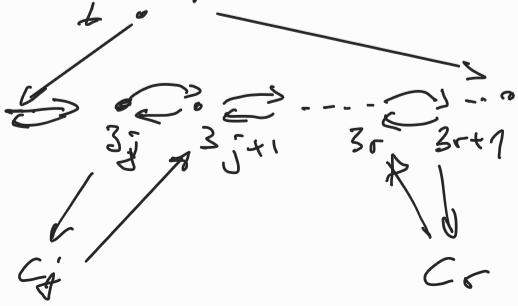
[Fall $x_j \in C_i$ ⇒ C_i = TRUE]

ist diese homogr. eindtg?

$x_i \in C_j$?
$\overline{x_i} \in C_j$

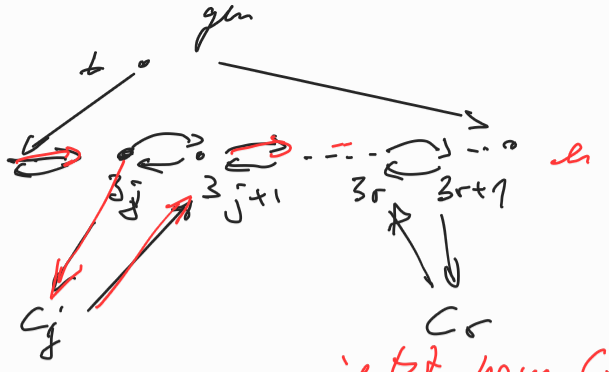
→ in G_i kann P nur eine Richtung einschlagen:



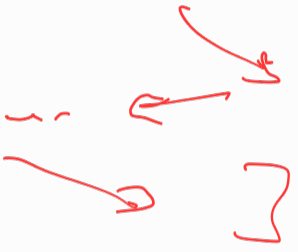


Sobald man sich für eine Richtung entschieden hat

Bspw:



jetzt kann C_r nicht mehr besucht werden & x_j wird nicht fälschlicherweise auf FALSE gesetzt.
(auch C_j andere Richtung



⇒ Zuweisung TRUE / FALSE eindeutig.
& jedes C_j hat mind 1 wahres Literal
& ist daher erfüllt.
⇒ φ erfüllbar.

Zuletzt noch: die Konstruktion benötige $O(K+\ell)$ Knoten & Kanten & kann daher in poly. Zeit in der größe von Input $|\varphi| \in O(\ell+k)$ ausgeführt werden

⇒ HAM_PFAD ist NP-schwer ∎

PROBLEM HAM_KREIS

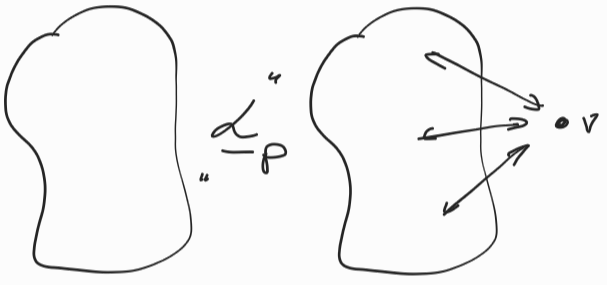
Input: gerichteter Graph $G = (V, E)$

Frage: Gibt es einen Hamilt. Kreis in G, d.h. eine Folge von Knoten $(v_1, \dots, v_n, v_1)$ od $(v_1, \dots, v_n)$ Ham. Pfad & $(v_n, v_1) \in E$?

Theorem HAM_KREIS ist NP-vollständig.

Beweis: $\in NP$ (ü Aufg) ✓

NP-schwer: HAM_PATH $\leq_p$ HAM_KREIS

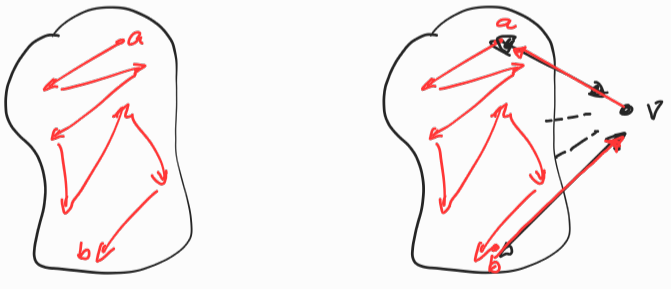


G Instanz HAM_PATH

Kopie von G + neuer Knoten v + Kanten $\{(u,v),(v,u)\}$ $\forall u$ aus G

$\Rightarrow$ Instanz G' von HAM_KREIS

geht in Poly. Zeit.



Ham. Pfad P

✓

□

Theorem: HAM_PFAD & HAM_KREIS bleiben NP-vollständig, selbst wenn der Input Graph ungerichtet ist (d.h. i.W. wenn $uv \in E$ dann $vu \in E$).

Beweis: ü Aufg. (hier geht 3SAT $\leq$ HAM_PFAD/KREIS nicht so direkt!)

Vollst. Graph $G=(V,E)$ ist unger. Graph sd $\forall u,v \in V$ m.? $u \neq v$: $uv \in E$.

Abk: $K_{|V|}$: K_1 •, K_2 •—•, K_3 △, K_4  , ...

TRAVELING-SALES-PUSON (TSP)
(dt. Problem d. Handlungsreisenden)

Input: vollständige Graph $G=(V,E)$ m.? kantengewichten $w: E \to \mathbb{R}_{\geq 0}$ & nat. Zahl k.

Frage: Gibt es ham. Kreis C in G sd $\sum_{e \in C} w(e) \leq k$?

Theorem: TSP ist NP-vollst.

Beweis: $\in NP$ (Übungsaufg)

NP-schwer: HAM_KREIS $\leq_p$ TSP.
für ungerichtete Graphen.

Sei G Instanz von HAM_KREIS

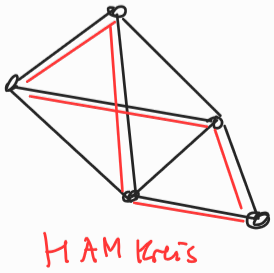
Betrachte $K_{|V(G)|}=(V(G),E)$ m.? $w: E \to \{1,2\}$

sd $w(e) = \begin{cases} 1, & e \in E(G) \\ 2, & e \notin E(G) \end{cases}$

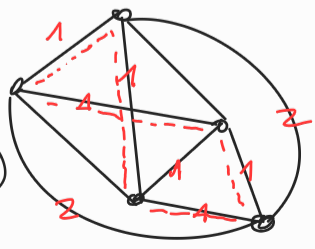
& $k = |V(G)|$

[Konstruktion gelt in polyzeit]

Bsp:

G 

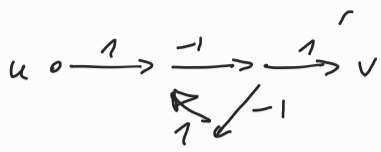
HAM Kreis

$(K_{|V(G)|}, w)$ 

"einfach": G hat Ham. Kreis $\Leftrightarrow$ G' hat Ham. Kreis d. Länge $k=|V(G)|$

Jetzt zu unserem ursprüngliche Problem.

beliebiger ungerichteter Graph $G=(V,E)$ mit bel. Kanten Gewichten.



$\Rightarrow$ $d(uv) = \infty$ möglich, da kein

Beschränkung auf einfache Pfade, d.h. kein Knoten kommt mehrmals vor.

$\Rightarrow$ im Bsp $d(uv) = 1$

kürzester einfacher Pfad (kEP)

Input: $G=(V,E)$ ungerichtet., $w: E \to \mathbb{R}$, $k \in \mathbb{Z}$.

Frage: Gibt es einfachen Pfad der Länge $\leq k$ in G?

Theorem: kEP ist NP vollständig.

$\in NP$. ($\hat{=}$ Aufg) ✓

NP-schwer: HAM-PFAD $\leq_P$ kEP
für ungr. graphen

G Instanz von HAM_PFAD

↓

Instanz von kEP: G, $w(e) = -1 \; \forall e \in E(G)$, $k = 1 - |V|$

G hat HAM-PFAD P $\Leftrightarrow$ # Kanten in $P = |V|-1$ (= spez. Baum)

$\Leftrightarrow$ $w(P) = \sum_{e \in P} w(e) = (|V|-1) \cdot (-1)$

$\Leftrightarrow$ $w(P) = 1 - |V| = k$

□

Zusammenfassung bisher:

- es gibt mehr Entscheidungsprobl. als Algorithmen
  ⇒ die meisten Probleme sind algor. nicht lösbar.
  (= sehr artifiziell")
- die Probleme in der Praxis sind i.d.R algorithmisch lösbar.
  Unterteilung: einfach (polyn. lösbar) vs. schwere Probl.

Algor. Dilemma wir möchten 3 wünschenswerte Eigenschaften von Algorithmen:

1. Löse NP-schwere Probleme
2. Laufe in Polyn. Zeit
3. Berechne exakte Lsg.

Wenn $P \neq NP$ ⇒ ein Alg. kann 2 aber nicht 3 dieser Eigenschaften besitzen.

2+3: polyn. Zeit, exakte (für Probleme in P)

1+2: Heuristiken, approx. Alg.

1+3: fixed parameter tractable Alg., ILP (FPT)
exponentielle Zeit exakte Alg.

Im folgenden werden wir Techniken zur Lösung von schweren Problemen betrachten: